

априорные алгоритмы

Apriori и **FP-Growth** — это два классических алгоритма для **анализа ассоциативных правил (association rule mining)**, которые используются для поиска частых наборов элементов (itemsets) и правил вида "если X, то Y" (например, "если покупают хлеб и молоко, то часто покупают и масло").



Apriori Algorithm

Основная идея:

Алгоритм использует свойство **антимонотонности (downward closure)**: если набор элементов не является частым, то все его надмножества тоже не могут быть частыми.

Как работает:

1. **Задаём минимальный порог поддержки (min_support)** — минимальная частота набора в данных.
2. **Находим все частые одиночные элементы (1-itemsets)**.
3. **Генерируем кандидатов большего размера (k-itemsets)** на основе частых наборов размера k-1.
4. **Проверяем поддержку** каждого кандидата и отбираем частые наборы.
5. **Повторяем** шаги 3-4, пока есть кандидаты.
6. **Генерируем ассоциативные правила** из частых наборов.

Пример (в продуктовой корзине):

```
Корзина 1: {молоко, хлеб}
Корзина 2: {молоко, хлеб, масло}
Корзина 3: {хлеб, масло}
Корзина 4: {молоко}
```

При min_support = 50%:

- Частые 1-itemsets: {молоко} (3 раза), {хлеб} (3 раза), {масло} (2 раза)
- Частые 2-itemsets: {молоко, хлеб} (2 раза), {хлеб, масло} (2 раза)
- Частые 3-itemsets: нет

Плюсы:

- Прост для понимания

- Хорошо работает на небольших данных

Минусы:

- Многократное сканирование базы данных
- Генерация большого числа кандидатов
- Медленный на больших данных

FP-Growth Algorithm

Основная идея:

Использует специальную структуру данных **FP-Tree (Frequent Pattern Tree)** для сжатого представления данных и поиска частых наборов без генерации кандидатов.

Как работает:

1. **Первое сканирование базы:** находим частые 1-itemsets и сортируем их по убыванию частоты.
2. **Строим FP-Tree:**
 - Каждый узел содержит элемент и счётчик
 - Узлы с одинаковым элементом связываются в список
 - Создаём таблицу заголовков для быстрого доступа
3. **Добываем частые наборы из FP-Tree:**
 - Для каждого элемента строим **условную базу паттернов (conditional pattern base)**
 - Строим **условный FP-Tree**
 - Рекурсивно добываем частые наборы

Пример FP-Tree:

Для данных:

```
1: {f, a, c, d, g, i, m, p}
2: {a, b, c, f, l, m, o}
3: {b, f, h, j, o}
4: {b, c, k, s, p}
5: {a, f, c, e, l, p, m, n}
```

После сортировки по частоте и построения дерева получаем сжатую структуру.

Плюсы:

- Обычно в 10-100 раз быстрее Apriori
- Требуется только 2 сканирования базы данных
- Не генерирует кандидаты
- Эффективен на больших данных

Минусы:

- Сложнее в реализации
- Требуется больше памяти для построения дерева
- Менее интуитивно понятен



Сравнение Apriori и FP-Growth

Критерий	Apriori	FP-Growth
Подход	Генерация и проверка кандидатов	Построение дерева и добыча паттернов
Сканирования базы	Многократные (для каждого k)	2 раза
Память	Умеренная	Больше (для дерева)
Скорость	Медленнее	Быстрее
Масштабируемость	Плохая	Хорошая
Популярность	Классический, учебный	Практический, промышленный



Практическое применение в рекомендательных системах:

В контексте ноутбука Otto:

Ковизитные матрицы в ноутбуке — это по сути **частые 2-itemsets**, найденные из истории пользователей. Они показывают, какие товары часто встречаются вместе в сессиях.

Пример использования:

Если алгоритм находит, что товары А и В часто встречаются вместе:

- При просмотре товара A → рекомендуем B
- При просмотре товара B → рекомендуем A

Реальные примеры:

1. **Amazon:** "Часто покупают вместе"
2. **Netflix:** "Похожие фильмы"
3. **Супермаркеты:** Размещение связанных товаров рядом
4. **Spotify:** Плейлисты на основе часто слушаемых вместе треков

Ключевые метрики для ассоциативных правил:

1. **Support (Поддержка):**
 $P(A \cap B)$ — как часто набор встречается
2. **Confidence (Достоверность):**
 $P(B|A) = \text{support}(A \cup B) / \text{support}(A)$ — если купили A, то с какой вероятностью купят B
3. **Lift (Лифт):**
 $\text{support}(A \cup B) / (\text{support}(A) \times \text{support}(B))$
 - 1: положительная корреляция
 - = 1: независимость
 - < 1: отрицательная корреляция

Пример кода (Python):

```
# Apriori
from mlxtend.frequent_patterns import apriori
frequent_itemsets = apriori(df, min_support=0.01, use_colnames=True)

# FP-Growth
from mlxtend.frequent_patterns import fpgrowth
frequent_itemsets = fpgrowth(df, min_support=0.01, use_colnames=True)

# Генерация правил
from mlxtend.frequent_patterns import association_rules
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1)
```

В ноутбуке Otto используется похожий принцип, но оптимизированный под конкретную задачу рекомендаций с учётом времени, типов событий и других факторов.